programmatically here (not relying on the file conventions covered earlier):

```
val configuration = ConfigFactory.load("combined")
val subApplACfg = configuration.getConfig("subApplA")      ❶
val config = subApplACfg.withFallback(configuration)       ❷
```

❶ Selects the subtree subApplA

❷ Adds a configuration as fallback

Now you can request the property `MyAppl.description` using `config.getString ("MyAppl.description")` and get the result `"Sub application"` because the description configuration value is set in the configuration at the highest level (`subApplA.MyAppl.description`). However, when you ask for `MyAppl.version`, you get the value `10`: the version configuration value isn't defined in the higher configuration (`subApplA.MyAppl`), so the normal fallback mechanism is used to provide the configuration value.

7.4 Configuration in tests

By default, the `ActorTestKit` loads the properties from the application-test.conf file if it exists in your project; otherwise, it follows the order you've already seen (shown again as figure 7.5). You can also directly use a string with `ConfigFactory.parseString` to describe the configuration to add or overwrite a property. This example overwrites `akka.loglevel`:

```
ConfigFactory.parseString("akka.loglevel = DEBUG")
```